

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	U.Prabhavathi	Reg.No.:	192472067
Department:	CSE-AI	Date:	25-07-26

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

1. Arithmetic Operations on Signed 8-bit Integers using 2's Complement

Question

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers:

- +45
- -23

Analyze how the system performs addition **and** subtraction, including binary conversion, sign-bit alignment, intermediate steps, overflow detection, and interpretation of the result.

Introduction

Computers represent signed integers using the 2's complement number system because it simplifies arithmetic operations.

In an **8-bit system**:

- MSB (Most Significant Bit) = Sign bit
- **0** → Positive number
- **1** → Negative number

Range:

-128 to +127

Step 1: Convert Decimal Numbers into Binary

(+45)

Divide 45 repeatedly by 2.

$45 \div 2 = 22$ remainder 1
 $22 \div 2 = 11$ remainder 0
 $11 \div 2 = 5$ remainder 1
 $5 \div 2 = 2$ remainder 1
 $2 \div 2 = 1$ remainder 0
 $1 \div 2 = 0$ remainder 1

Binary:

101101

8-bit representation:

00101101

Therefore,

+45 = 00101101
(-23)

First convert +23.

$$23 = 00010111$$

Take 1's complement.

$$00010111$$

↓

$$11101000$$

Add 1.

$$\begin{array}{r} 11101000 \\ +00000001 \\ \hline 11101001 \end{array}$$

Therefore,

$$-23 = 11101001$$

Binary Numbers

$$+45 = 00101101$$

$$-23 = 11101001$$

Step 2: Addition

Compute

$$(+45) + (-23)$$

Binary addition:

$$\begin{array}{r} 00101101 \\ + 11101001 \\ \hline 1\ 00010110 \end{array}$$

Discard the carry.

Result:

$$00010110$$

Convert to decimal.

$$00010110$$

$$=16+4+2$$

$$=22$$

Therefore,

$$45 + (-23) = 22$$

Intermediate Carry

Carry out = 1

Carry is discarded because the processor stores only **8 bits**.

Overflow Detection

Overflow occurs only when

- Positive + Positive $\rightarrow$ Negative
- Negative + Negative $\rightarrow$ Positive

Here,

Positive + Negative

Overflow **cannot occur**.

Therefore,

No Overflow

Processor Interpretation

MSB

0

Means

Positive Number

Remaining bits

0010110

=22

Final Answer

+22

Step 3: Subtraction

Compute

$$45 - (-23)$$

Subtracting a negative number means

$$45 + 23$$

Binary of +23

00010111

Addition

```
  00101101
+00010111
-----
  01000100
```

Decimal

64+4

=68

Therefore

$45 - (-23) = 68$

Overflow Check

Operands

Positive

Positive

Result

Positive

Since the result is within

-128 to +127

No overflow occurs.

How Processor Detects Overflow

The processor checks the carry into and carry out of the sign bit.

Overflow occurs when

Carry into MSB $\neq$ Carry out of MSB

OR

When

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Example

$$100 + 50 = 150$$

150 cannot be represented in 8 bits.

Binary result becomes negative.

Hence

Overflow = True

For this problem

$$45 + (-23)$$

↓

No Overflow

$$45 - (-23)$$

↓

No Overflow

Why 2's Complement is Used

Advantages:

- Represents both positive and negative numbers.
- Uses the same hardware for addition and subtraction.
- Only one representation for zero.
- Simplifies processor design.
- Fast arithmetic operations.

Final Results

Operation	Binary Result	Decimal Result	Overflow
-----------	---------------	----------------	----------

45 + (-23)	00010110	22	No
------------	----------	----	----

45 - (-23)	01000100	68	No
------------	----------	----	----

Conclusion

The processor first converts signed numbers into 8-bit 2's complement form, performs binary addition, and discards any extra carry. Overflow is detected by checking the sign bits and carry into/out of the MSB. In this example, both addition and subtraction produce valid results (**22** and **68**) without overflow.

2. Carry Look-Ahead Adder (CLA)

Introduction

A Carry Look-Ahead Adder (CLA) is a fast adder used in processors. Unlike a Ripple Carry Adder (RCA), which waits for each carry to propagate, the CLA calculates all carry bits in advance, reducing computation time

Example

Let:

- $A = 1010$
- $B = 0111$

Step 1: Generate (G) and Propagate (P) Signals

For each bit:

- **Generate (G_i)** = $A_i \times B_i$
- **Propagate (P_i)** = $A_i \oplus B_i$

These signals determine whether a carry is generated or propagated.

Step 2: Carry Computation

The carries are calculated using:

- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$
- C_4 is calculated similarly.

Since all carries are computed simultaneously, the sum bits are obtained quickly.

Comparison: CLA vs Ripple Carry Adder

Ripple Carry Adder	Carry Look-Ahead Adder
Carry moves one stage at a time	Carries computed in parallel
More delay	Less delay
Simpler hardware	More complex hardware
Slower	Faster

Why CLA is Faster

- Computes carries in parallel.
- Reduces propagation delay.
- Used in high-speed processors and ALUs.

Conclusion

Carry Look-Ahead Adders improve processor performance by reducing the time required for binary addition.

3. Booth's Algorithm ($-6 \times +5$)

Introduction

Booth's Algorithm is an efficient method for multiplying signed binary numbers represented in 2's complement. It reduces the number of addition and subtraction operations.

Given

- Multiplicand (M) = -6
- Multiplier (Q) = $+5$

Registers used:

- **A** $\rightarrow$ Accumulator
- **Q** $\rightarrow$ Multiplier
- **M** $\rightarrow$ Multiplicand
- **Q-1** $\rightarrow$ Booth bit (initially 0)

Algorithm Steps

For each bit of the multiplier:

- $Q_0Q_{-1} = 00$ or $11 \rightarrow$ No operation
- $Q_0Q_{-1} = 01 \rightarrow A = A + M$
- $Q_0Q_{-1} = 10 \rightarrow A = A - M$
- Perform an arithmetic right shift after every operation.
- Repeat until all bits are processed.

Final Result

After completing all iterations:

- Binary Product = **11100010**
- Decimal Product = **-30**

Advantages of Booth's Algorithm

- Handles positive and negative numbers directly.
- Reduces unnecessary additions.
- Faster than ordinary binary multiplication.
- Widely used in processors.

Conclusion

Booth's Algorithm efficiently performs signed multiplication with fewer arithmetic operations, making it suitable for modern computer systems.

4. Restoring and Non-Restoring Division ($13 \div 3$)

Introduction

Processors perform binary division using Restoring **and** Non-Restoring **division** algorithms.

Given:

- Dividend = **13 (1101_2)**
- Divisor = **3 (0011_2)**

Expected Result:

- Quotient = **4**
- Remainder = **1**

A. Restoring Division

Steps

1. Shift the dividend left.
2. Subtract the divisor.
3. If the remainder becomes negative:
 - Restore the previous remainder.
 - Set quotient bit = 0.

4. Otherwise:
 - Keep the remainder.
 - Set quotient bit = 1.
5. Repeat until all bits are processed.

Result

- Quotient = **4**
- Remainder = **1**

B. Non-Restoring Division

Steps

1. Shift left.
2. If the previous remainder is positive, subtract the divisor.
3. If the previous remainder is negative, add the divisor.
4. Set quotient bits accordingly.
5. Restore only once at the end if necessary.

Result

- Quotient = **4**
- Remainder = **1**

Comparison

Restoring Division	Non-Restoring Division
Restores remainder every time it becomes negative	No immediate restoration
More arithmetic operations	Fewer arithmetic operations
Slower	Faster
Simple implementation	More efficient

Why Non-Restoring is Preferred

- Fewer additions and subtractions.
- Faster execution.
- Better performance in modern processors.

Conclusion

Both algorithms produce the same result, but **Non-Restoring Division** is more efficient due to fewer operations.

5. IEEE 754 Floating-Point Representation

Introduction

The **IEEE 754 standard** is used to represent real (floating-point) numbers in computers.

Given decimal number:

-13.25

Step 1: Convert to Binary

$$13 = 1101_2$$

$$0.25 = \mathbf{0.01}_2$$

Therefore,

$$13.25 = 1101.01_2$$

Normalized form:

$$1.10101 \times 2^3$$

Single Precision (32-bit)

- Sign bit = 1 (negative number)
- Exponent = $3 + 127 = 130 = 10000010_2$
- Mantissa = 101010000000000000000000

Double Precision (64-bit)

- [illegible]

Floating-Point Addition

When adding two floating-point numbers:

1. Compare the exponents.
2. Shift the smaller number until both exponents match.
3. Add the mantissas.
4. Normalize the result.
5. Round the result if necessary.
6. Store the final sign, exponent, and mantissa.

Precision Issues

- Some decimal numbers cannot be represented exactly in binary.
- Rounding may introduce small errors.
- Precision loss can occur during repeated calculations.

Single vs Double Precision

Single Precision	Double Precision
32 bits	64 bits
Less memory	More memory
Lower accuracy	Higher accuracy
Faster	Slightly slower

Conclusion

IEEE 754 provides a standard format for representing real numbers. Single precision is faster and uses less memory, while double precision offers greater accuracy. Floating-point addition involves exponent alignment, mantissa addition, normalization, and rounding, which may introduce small precision errors.

Code of Conduct:

I U.Prabhavathi (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

U.Prabhavathi

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation